I. INTRODUCTION TO SCIKIT-LEARN (SKLEARN)

Scikit-Learn is the most robust and widely used Python library for classical Machine Learning. Built on NumPy, SciPy, and Matplotlib, it provides a consistent API for a vast range of supervised and unsupervised protective modeling tasks. In a RAG system, understanding Sklearn is essential for tasks like document classification, clustering, and dimensionality reduction.

1.1. The "Estimator" Design Pattern

All objects in Scikit-Learn share a uniform interface:

- **fit(X, y):** The training step where the model learns patterns from data.
- **predict(X):** The inference step where the model applies learned patterns to new data.
- **transform(X):** Used in preprocessing to modify data (e.g., scaling or encoding).

II. SUPERVISED LEARNING: REGRESSION AND CLASSIFICATION

2.1. Linear and Logistic Regression

- **Linear Regression:** Predicting a continuous value (e.g., predicting a student's final GPA based on study hours).
- **Logistic Regression:** Despite its name, it is used for **Classification** (e.g., predicting if a student will "Pass" or "Fail"). It uses the Sigmoid function to output a probability between 0 and 1.

2.2. Tree-Based Models (Decision Trees & Random Forests)

Decision Trees split data based on feature values to reach a conclusion. However, they are prone to **Overfitting** (learning the noise instead of the signal).

- **Random Forest:** An "Ensemble" method that builds hundreds of decision trees and averages their results to increase accuracy and stability.

2.3. Support Vector Machines (SVM)

SVM finds the "Hyperplane" that best separates different classes with the maximum margin. It is highly effective for high-dimensional data like text classification.

III. UNSUPERVISED LEARNING: FINDING HIDDEN PATTERNS

3.1. K-Means Clustering

Used to group unlabeled data into K clusters. In Big Data, this is used for **Customer Segmentation** or grouping similar documents in a database.

- **Elbow Method:** A statistical technique used to find the optimal number of clusters (K).

3.2. Principal Component Analysis (PCA)

A dimensionality reduction technique. PCA compresses a dataset with 100 features into 2 or 3 "Principal Components" while retaining most of the variance. This speeds up model training and allows for data visualization.

IV. MODEL EVALUATION AND VALIDATION

4.1. The Train/Test Split

To ensure a model generalizes well to unseen data, we must never test on the same data used for training. Typically, an 80/20 or 70/30 split is used.

4.2. Classification Metrics (The Confusion Matrix)

Accuracy is often misleading, especially in imbalanced datasets. We use:

- **Precision:** How many predicted "Spams" were actually spam?
- **Recall:** How many actual "Spams" did the model catch?
- **F1-Score:** The harmonic mean of Precision and Recall.

4.3. Cross-Validation (K-Fold)

Instead of a single split, K-Fold divides the data into K parts, training K times to ensure the model's performance is consistent across the entire dataset.

V. PRACTICAL IMPLEMENTATION: THE ML PIPELINE

Python

```
from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import classification_report

from sklearn.preprocessing import StandardScaler


# 1. Preprocessing: Scaling features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


# 2. Splitting data

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2)


# 3. Training

model = RandomForestClassifier(n_estimators=100)

model.fit(X_train, y_train)
```

```
# 4. Evaluation
predictions = model.predict(X_test)
print(classification_report(y_test, predictions))
```